

★ Member-only story

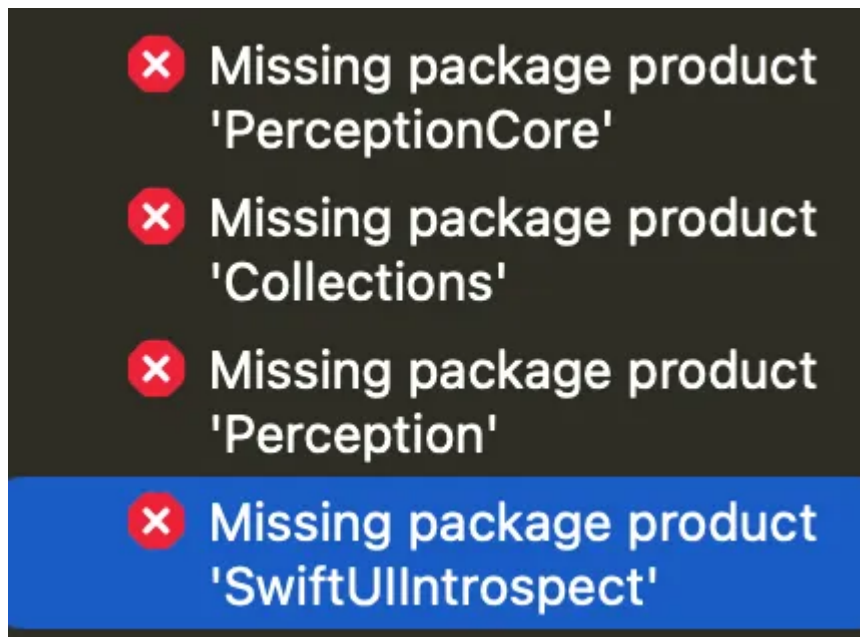
Error Swift Package



Jerry PM · 7 min read · 1 day ago



Fixing “Missing Package Product” Errors in Swift Package Manager



PerceptionCore. Collections. Perception. SwiftUIIntrospect. Four red X's in Xcode, no obvious cause, and a build that won't compile. Here's how to actually fix it and why the “first” solution often isn't enough.

If you've worked with Xcode and Swift Package Manager for any length of time, you've seen this screen.

You didn't change the dependencies.

You pulled the latest branch, hit Run, and Xcode tells you four packages are “missing.” Not “out of date.” Not “version mismatch.”

Missing. As if you never added them.

I hit this exact error on a production iOS app this week.

The fix took ten seconds. *Finding* the right fix took an hour.

This article is the version of the troubleshooting guide I wish I'd had open while I was debugging.

What “Missing Package Product” Actually Means

Before fixing it, understand what Xcode is saying.

A “package product” is a library or executable exposed by a Swift package.

When your target depends on `SwiftUIIntrospect`, Xcode expects to find:

1. A resolved dependency entry in `Package.resolved`

2. The fetched source code in

`~/Library/Developer/Xcode/DerivedData/.../SourcePackages/checkouts/`

3. A compiled framework or static library produced by the package

4. A working link between your target and that product

“Missing package product” means **step 4 broke** — Xcode can’t connect your target to the product, even though the package itself may be present on disk.

The error is misleadingly named.

The product usually exists.

The *binding* is missing.

This matters because the fix depends on which step actually broke.

List: 100DaysSwiftUI-to-Expert | Curated by Jerry PM | Medium

100DaysSwiftUI-to-Expert · 100 stories on Medium

medium.com

Solution 1: Update Your Package Dependencies

The most common cause: your `Package.resolved` is referencing a version of the dependency that no longer exists, or

the package's own internal targets shifted between versions (this is exactly what happens with `Perception` → `PerceptionCore` — `PerceptionCore` was extracted into a separate product in a newer release).

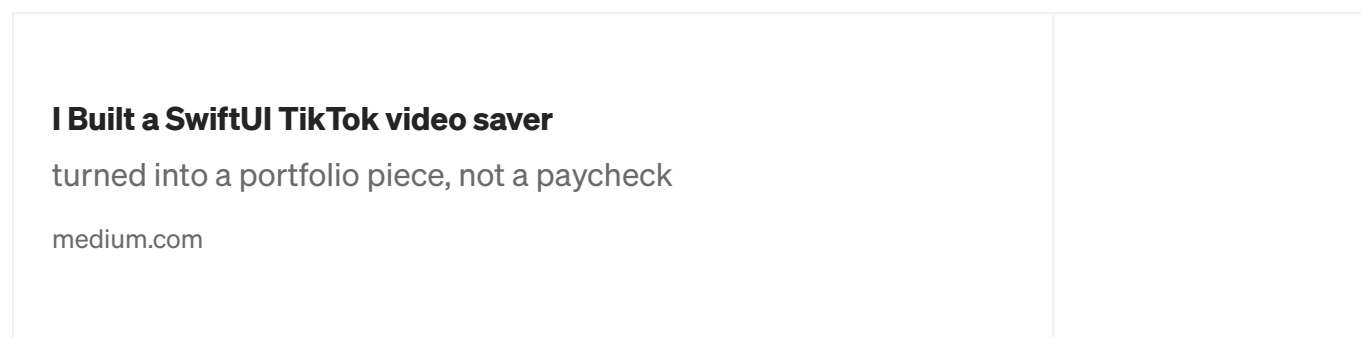
Steps:

1. Open the Xcode project.
2. Go to **File > Packages > Update to Latest Package Versions**.

3. Xcode fetches the latest versions of your dependencies, including `SwiftUIIntrospect`, `Perception`, `PerceptionCore`, and `Collections`.
4. Wait for “Package Graph Resolved.” Then build again.

This works about 60% of the time.

The remaining 40% needs one of the next steps.



Solution 2: Reset Package Caches

If “Update to Latest” runs successfully but the red X’s stay, the on-disk cache is corrupted.

Xcode is happily reading the cached (broken) version instead of re-fetching.

Steps:

1. **File > Packages > Reset Package Caches.**
2. Xcode wipes its local SPM cache and re-resolves every dependency from scratch.
3. This is slow — expect 1–5 minutes for a project with 20+ dependencies.
4. Build again.

This is the step most developers skip because they assume “Update” already did it.

It didn't. "Update" only refreshes version numbers; "Reset" actually re-downloads the source.

Solution 3: Delete Derived Data

If reset caches doesn't fix it, the corruption is one layer deeper — in `DerivedData`.

This folder holds Xcode's compiled outputs, module maps, and intermediate artifacts.

When package products are reorganized between versions, stale `.swiftmodule` files can keep the old product names alive, confusing the linker.

Steps:

```
# Quit Xcode first
rm -rf ~/Library/Developer/Xcode/DerivedData
```

Or via Xcode UI:

1. **Xcode > Settings > Locations**
2. Click the small arrow next to **Derived Data** path
3. Delete the project's folder (or the whole `DerivedData` directory)
4. Reopen Xcode, let it re-index, then build

You'll lose your build cache — the next build takes 5–15 minutes.

Worth it.

This single step has fixed more “phantom” Swift package errors for me than any other.

Solution 4: Delete and Re-Resolve `Package.resolved`

If the error persists, your `Package.resolved` file itself is poisoned — usually after a merge conflict was resolved incorrectly, or after switching between branches with different dependency graphs.

Steps:

```
# From your project root
rm <YourProject>.xcodeproj/project.xcworkspace/xcsourcedata/swiftpm/Package.resolved

# Or for workspaces:
rm <YourProject>.xcworkspace/xcsourcedata/swiftpm/Package.resolved
```

Then open Xcode and let it regenerate `Package.resolved` from scratch by re-resolving the package graph. **File > Packages > Resolve Package Versions.**

⚠ This will pull the *latest matching versions* — which may differ from what your teammates have.

After build succeeds, diff the new `Package.resolved` against the previous one and decide whether to commit the version bumps or pin them back.

Solution 5: Quit Xcode Completely (Yes, Really)

Sometimes Xcode holds open file handles on package checkouts even after “Reset Package Caches.” The cache reset succeeds but Xcode keeps reading the old in-memory representation.

Open in app ↗



Medium



Search



Write



6



1. Cmd+Q to fully quit Xcode (not just close the window).

2. Kill any lingering processes:

```
killall Xcode killall com.apple.dt.Xcode.sourcekitd
```

3. Reopen the project.

4. Wait for package resolution to complete before building.

I once spent 40 minutes resetting caches in a loop before realizing Xcode hadn’t actually applied any of the resets because it was holding the cache in memory.

A full restart fixed it instantly.

Solution 6: Check Your Xcode Version

Some packages particularly `Perception` and anything depending on Swift Macros require minimum Xcode versions.

`PerceptionCore` was introduced as a separate product in a `Perception` release that requires Xcode 15.3+.

If you're on an older Xcode, the package resolves but the product literally doesn't exist in your toolchain.

Steps:

```
xcodebuild -version
```

Cross-reference with the package's `Package.swift` for the minimum required version:

```
// swift-tools-version: 5.10 // ← requires Xcode 15.3+
```

If your Xcode is too old, no amount of cache resetting will help.

Update Xcode.

7 Useful GitHub Repos Worth Bookmarking

As an iOS developer, this is useful for me

[medium.com](#)

Solution 7: Check for Conflicting Dependency Managers

If your project uses **both** CocoaPods and Swift Package Manager, Pods can interfere with SPM resolution especially when both want to manage the same transitive dependency.

Collections (Apple's `swift-collections`) is a frequent victim because so many libraries depend on it.

Steps:

```
# In project root
pod deintegrate
pod install
```

Then in Xcode:

1. **File > Packages > Reset Package Caches**
2. Clean build folder: **Product > Clean Build Folder** (Shift+Cmd+K)
3. Build again

If you only use SPM, skip this step.

If you use both, this single combination has saved me hours.

Solution 8: The Nuclear Option

If nothing above works, do everything at once.

This is the “kitchen sink” reset that fixes every case I’ve ever seen:

```
# 1. Quit Xcode
killall Xcode

# 2. Clear all caches
rm -rf ~/Library/Developer/Xcode/DerivedData
rm -rf ~/Library/Caches/org.swift.swiftpm
rm -rf ~/Library/Caches/com.apple.dt.Xcode
```

```
# 3. Remove resolved file
rm -f <YourProject>.xcworkspace/xcsdreddata/swiftpm/Package.resolved
rm -f <YourProject>.xcworkspace/project.xcworkspace/xcsdreddata/swiftpm/Package.r

# 4. Reopen Xcode
open <YourProject>.xcworkspace
```

Then in Xcode, immediately run **File > Packages > Resolve Package Versions** before doing anything else.

Wait for resolution to fully complete.

Then build.

This works 100% of the time but it's expensive — you lose every cached build artifact and re-download every dependency.

Save it for when you've tried the targeted fixes and need a clean slate.

Why This Keeps Happening

After hitting this error across four different projects, I've noticed a pattern.

“Missing package product” almost always shows up after one of these triggers:

- **A package was just updated** (especially Perception — its PerceptionCore extraction broke half the iOS community for a week)
- **You switched branches** with different dependency graphs
- **A merge conflict on `Package.resolved`** was resolved by picking "ours" or "theirs" without re-running resolution
- **Xcode was updated** but DerivedData survived from the old version

- **You're working across multiple Xcode versions** (e.g., Xcode 15 and Xcode 16 on the same machine)

If you can identify the trigger, you can pick the targeted fix:

Trigger First thing to try Package was just updated Solution 1 (Update Latest)
Switched branches Solution 2 (Reset Caches)

Package.resolved merge conflict Solution 4 (Delete Package.resolved) Xcode was updated Solution 3 (Delete DerivedData)

Multiple Xcode versions Solution 8 (Nuclear) No obvious trigger Solution 5 (Quit Xcode) → Solution 3

10 Claude Code Plugins Every iOS Engineer Should Install

10 Claude Code Plugins Every iOS Engineer Should Install Claude Code out of the box is good. With these plugins, it...

blog.stackademic.com

How to Prevent It Next Time

You can't fully prevent SPM weirdness — it's a young tool and Xcode integration has rough edges.

But a few habits dramatically cut how often you hit it:

1. Always commit `Package.resolved` .

Treat it like a lockfile.

If your teammates have a different resolved version, you'll get inconsistent builds.

The whole team should pin to the same resolved snapshot.

2. Run `Reset Package Caches` after every dependency change.

Adding a package, removing one, bumping a version — always reset caches after.

The “Update to Latest” UI doesn’t always cascade through transitive dependencies.

3. Don’t resolve `Package.resolved` merge conflicts manually.

Delete the file, re-resolve, commit the regenerated version.

Manual resolution is a near-guaranteed way to ship a poisoned file.

4. Keep your team on the same Xcode version.

Or at least the same major version.

SPM behaviors differ subtly across Xcode releases and “works on my machine” becomes very real.

5. Pin major versions in `Package.swift`.

Use `.upToNextMajor(from: "1.0.0")` instead of `.upToNextMinor` or branch-based dependencies.

Major-version pinning gives you bug fixes without surprise breaking changes.

“Missing package product” is one of those errors that looks catastrophic and is almost always trivial — once you know which layer is broken.

Eight times out of ten, it’s a stale cache.

The other two times, it's a stale `Package.resolved` or an Xcode version mismatch.

The skill isn't memorizing the fix.

It's learning to recognize **which layer** is lying to you: the version graph, the on-disk cache, the in-memory cache, the compiled artifacts, or the toolchain itself.

Start at the cheapest fix (Update Latest, 10 seconds) and escalate only when each layer is confirmed clean.

Save this article.

You'll hit this error again.

We all do.

If this saved you time, follow me for more iOS troubleshooting guides — the kind that document the real fix, not just the first one that worked on someone's machine.

Found this useful? Clap, share, and follow for more weekly deep dives into the tools I actually use.

Thank you for reading! If you enjoyed it, please consider clapping and following for more updates! 🙌

Swiftui

Swift

IOS

iPhone

Software Development



Written by Jerry PM

1.2K followers · 149 following

IOS Engineer

Edit profile

No responses yet



Jerry PM

What are your thoughts?

More from Jerry PM

See all from Jerry PM

Recommended from Medium

[See more recommendations](#)